

Machine Learning Engineering (PCAM* ZC412)

Mini-Project Assignment Brief — EC-1 Project (Weight: 40% | Duration: 28 Days)

1. Overview

This mini-project requires you to design, build, and operate an end-to-end machine learning pipeline — from raw data to a monitored, deployed service. You will choose ONE of three problem statements (Flavor A, B, or C) based on your interest. All three flavors are of equal complexity, follow the same 4-week structure, map to the same course modules (M1 through M5), and are evaluated using the same rubric — only the input data and modeling task differ.

2. Objectives

- Design and implement a reliable data ingestion, validation, and feature engineering pipeline.
- Apply engineering discipline to ML experimentation through tracking, comparison, and reproducibility.
- Package and deploy a trained model as a production-style service with a REST API.
- Implement monitoring, simulate drift, and design a retraining strategy for a deployed model.
- Demonstrate end-to-end ownership of the ML system lifecycle, consistent with course outcomes LO1–LO5.

3. Choose Your Flavor

Select one of the following problem statements. All three are equally weighted and equally complex; choose based on the domain that interests you most.

Flavor A — Delivery / Ride ETA Prediction

Domain: Tabular data | Modules: $M2 \rightarrow M3 \rightarrow M4 \rightarrow M5$

Problem Statement: A logistics or ride-hailing platform wants to predict delivery time (or ride ETA) based on trip distance, time of day, weather conditions, traffic patterns, and pickup/drop-off location. Build an end-to-end ML pipeline that ingests trip data, engineers time- and location-based features, trains and compares models to predict ETA, deploys the best model as a service, and monitors it for accuracy drift as traffic and seasonal patterns change.

Week	Module	Task
Week 1	M2	Ingest historical trip data; validate schema (missing GPS pings, invalid timestamps); engineer features (hour-of-day, weekday/weekend, distance, weather); version the dataset.
Week 2	M3	Train and compare models (e.g., linear regression vs. gradient boosting); track experiments and hyperparameters.
Week 3	M4	Package the best model; serve via a REST API that accepts trip details and returns predicted ETA.
Week 4	M5	Log predictions vs. actual times; simulate drift (e.g., festival/rush-hour surge); set up monitoring and a retraining trigger.

Suggested Dataset(s): NYC Taxi Trip Duration (Kaggle), or a synthetic delivery dataset with distance/weather/time features.

Flavor B — Image-Based Defect / Quality Classifier

Domain: Vision data | Modules: $M2 \rightarrow M3 \rightarrow M4 \rightarrow M5$

Problem Statement: A manufacturing or quality-assurance team wants to automatically flag defective products from images captured on the production line. Build an end-to-end ML pipeline that ingests and preprocesses product images, trains a classifier to distinguish defective from non-defective items, deploys it as an inference service, and monitors performance as new product variants or lighting conditions appear.

Week	Module	Task
Week 1	M2	Ingest and validate image data; build a preprocessing/augmentation pipeline; version the dataset.
Week 2	M3	Train and compare CNN/transfer-learning models; track experiments and reproducibility.
Week 3	M4	Package the model; serve via a REST API accepting image uploads; handle malformed/edge-case inputs.
Week 4	M5	Log predictions and confidence scores; simulate distribution shift (lighting/angle changes); monitor and design a retraining workflow.

Suggested Dataset(s): Casting Product Image Data for Quality Inspection (Kaggle), or Surface Defect Detection datasets.

Flavor C — Support Ticket / Review Sentiment Classifier

Domain: NLP / text data | Modules: $M2 \rightarrow M3 \rightarrow M4 \rightarrow M5$

Problem Statement: A customer support or e-commerce platform wants to automatically classify incoming text — support tickets by urgency/intent, or product reviews by sentiment. Build an end-to-end ML pipeline that ingests raw text, engineers features/embeddings, trains a classifier, deploys it as a service, and monitors for concept drift as language and topics evolve.

Week	Module	Task
Week 1	M2	Ingest raw text data; clean and tokenize; build a feature pipeline (TF-IDF/embeddings); version the dataset.
Week 2	M3	Train and compare models (classical ML vs. a fine-tuned transformer); track experiments.
Week 3	M4	Package the model; serve via a REST API accepting text input; handle edge cases (empty/malformed text).
Week 4	M5	Log predictions; simulate concept drift (new slang/topics); monitor performance and design retraining triggers.

Suggested Dataset(s): Amazon/Yelp Reviews, Twitter Sentiment140, or a support-ticket intent dataset.

4. General Expectations

- Work in groups as mentioned in the group registration spreadsheet.
- Use open-source tools only (e.g., MLflow/DVC for tracking and versioning, FastAPI/Flask for serving, Docker for packaging).
- Code must be version-controlled with meaningful, incremental commits reflecting weekly progress — not a single last-day upload.
- All design decisions (model choice, drift-detection approach, retraining trigger) must be justified in the documentation, not just implemented.

- Academic integrity applies: pipelines and code must be your own work; third-party libraries and datasets must be cited.

5. Weekly Timeline & Milestones

Milestone	Expectation
Day 1–2	Flavor selection confirmed; repository initialized.
End of Week 1	Data ingestion, validation, and feature pipeline complete; dataset version tagged.
End of Week 2	At least two tracked experiments completed; best model identified with justification.
End of Week 3	Model packaged and deployed; API endpoint tested with sample inputs.
End of Week 4 (Day 28)	Monitoring/drift simulation complete; final submission package and demo ready.

6. Submission Checklist

No.	Deliverable
1	Versioned dataset and pipeline code (GitHub/GitLab repository link, with commit history reflecting weekly progress).
2	Experiment tracking logs and a short model comparison report (screenshots or exported logs, e.g., MLflow).
3	Deployed model with a working API endpoint, including sample request/response test calls (e.g., Postman collection or curl commands).
4	Monitoring log, drift-simulation report, and a documented retraining trigger design.
5	README with architecture diagram, setup instructions, and a 5–7 minute recorded/live demo or presentation.

7. Evaluation Rubric (Total: 40% of Course Grade)

Evaluation Criterion	Weight	What Is Assessed
Data Engineering & Versioning (M2)	20%	Data ingestion and validation logic, quality of feature engineering, correct use of dataset versioning (e.g., DVC/Git).
Experimentation & Reproducibility (M3)	20%	Multiple tracked experiments, clear comparison of model runs, ability to reproduce a chosen run from logged configuration.
Model Packaging & Deployment (M4)	20%	Correct packaging/serialization, working REST API, input validation and error handling, basic latency/throughput awareness.
Monitoring, Drift & Retraining (M5)	20%	Prediction logging, realistic drift simulation, meaningful monitoring signals, and a sound retraining trigger design.
Documentation & Presentation	20%	Clarity of README/architecture diagram, code organization, and quality of final demo/presentation.

8. Academic References

- T1: Machine Learning Production Systems, Robert Crowe, et al.; O'Reilly, 2024.
- T2: Machine Learning Engineering, Andriy Burkov, 2020.
- R1: Machine Learning Engineering with Python (2nd Edition), Andrew P. McMahon, Packt, 2023.

Note: This brief should be read alongside the official course handout for Machine Learning Engineering (PCAM* ZC412), particularly Part B (Learning Plan) and the Evaluation Scheme, for full context on module references and evaluation policies.